National University
of computer and emerging sciences

Project: Metro App

GROUP MEMBERS:

1. Muhammad Ashir (22K-4504)
2. Muhammad Anas (22K-4639)
3. Muhammad Ammar (22K-4575)

Section: BCS-5G

Teacher: Miss Urooj

1. Introduction:

The Metro App addresses the increasing need for streamlined metro navigation in urban environments. The project's primary goal is to develop a reliable and intuitive application that allows users to manage metro stations and plan their routes seamlessly.

3. Project Scope:

The Metro App encompasses the following features:

- Addition and removal of metro stations.
- Calculation of the shortest path based on the number of stops.
- User-friendly interface for enhanced user experience.

4. Methodology:

The project utilized C++ programming language along with the SFML (Simple and Fast Multimedia Library) for creating graphical user interfaces.

Algorithms/Data Structures/Methods used:

- Trie Method for efficient searching.
- Queue Data Structure.
- Astar’s Search Algorithm.
- Thread Library for optimization.
- SFML for Graphic User Interface
- Binary Search for efficient searching of stations stored in a BST.
- Binary Search Tree for storing stations.
- Graph Data Structure to represent the stations.
- Pair Method to combine two data types.

5. System Architecture:

The system architecture comprises three main components: the user interface, the application logic, and the data storage. These components interact seamlessly to provide users with a robust and responsive metro navigation experience.

6. Features:

The Metro App includes the following features:

- **Addition and Removal of Stations:** Users can dynamically manage metro stations.
- **Shortest Path Calculation:** The application computes the shortest path based on the number of stops, by using Euclidean Distance Formula.
- **Show Journey:** Users can view their travel journey, which includes their previous journeys travelled as well.

8. User Interface:

The user interface is designed with simplicity and functionality in mind. Users can interact with the app through an intuitive graphical interface, making it accessible for a wide range of users. The library used for the graphic user interface is “SFML”. Snapshots are provided below.

9. References:

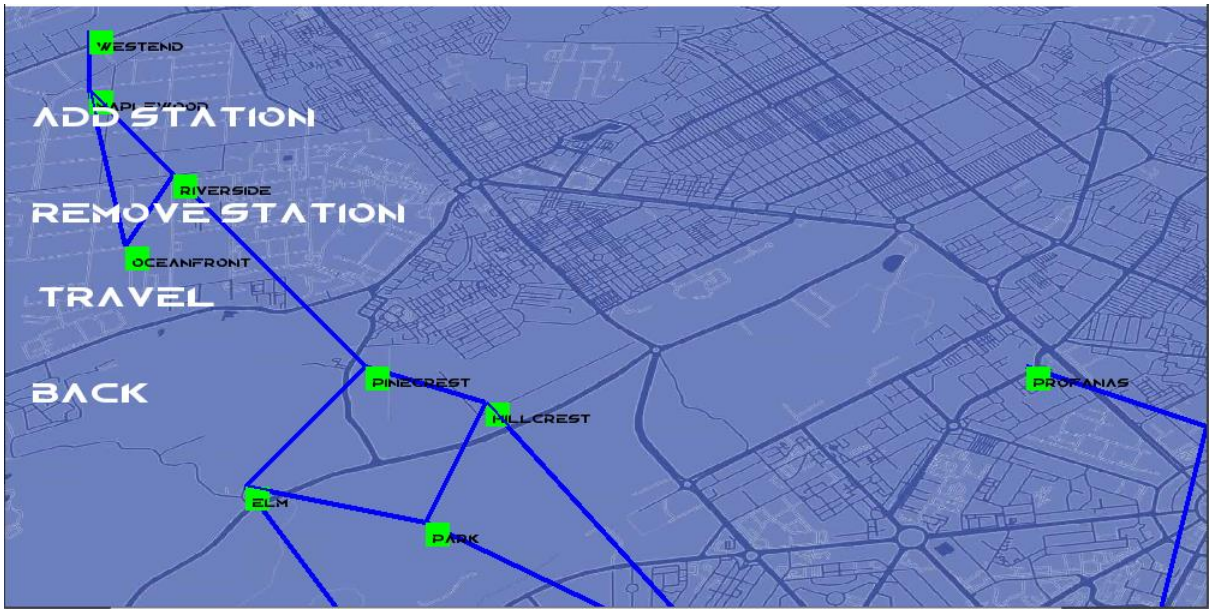
- SFML Documentation: SFML
- Astar’s Algorithm: GFG
- Threads: PDC BOOK

10. Working:

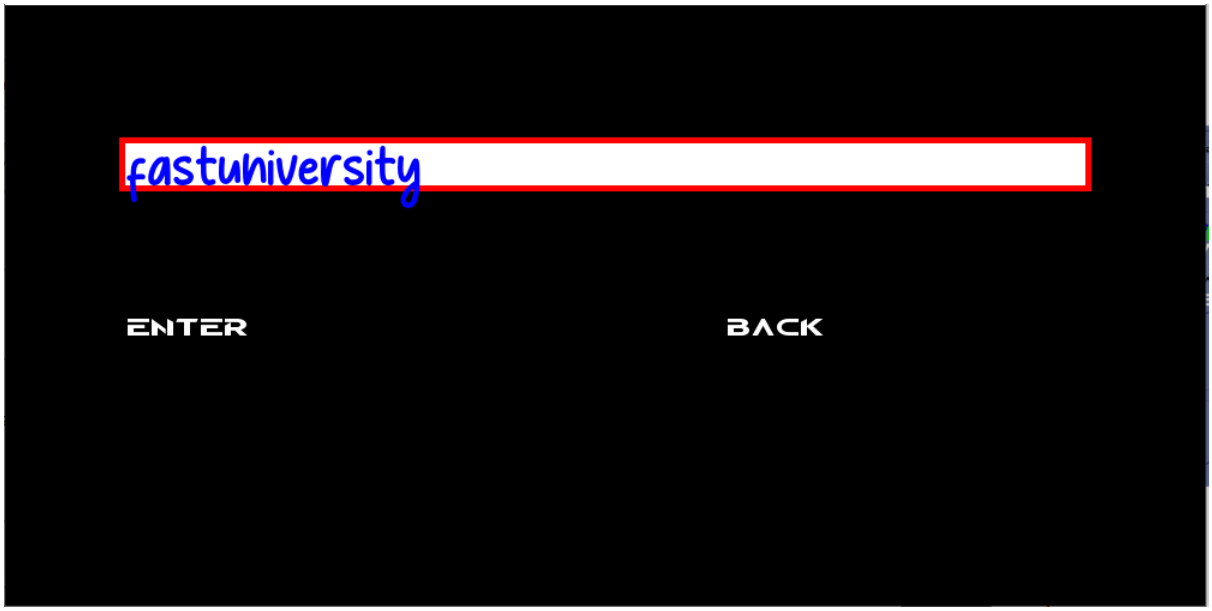
The Program initially logs you in to the app, allowing you to selection the following options:

- **Add Station:** You can add a station, if it is not showing in your map.
- **Remove Station:** You can remove a station, if you are not likely to visit there.
- **Travel:** Allows you to enter into the second menu, which then asks whether you’d like to see your previous journey, or if you want to travel today to a certain destination.

Menu:

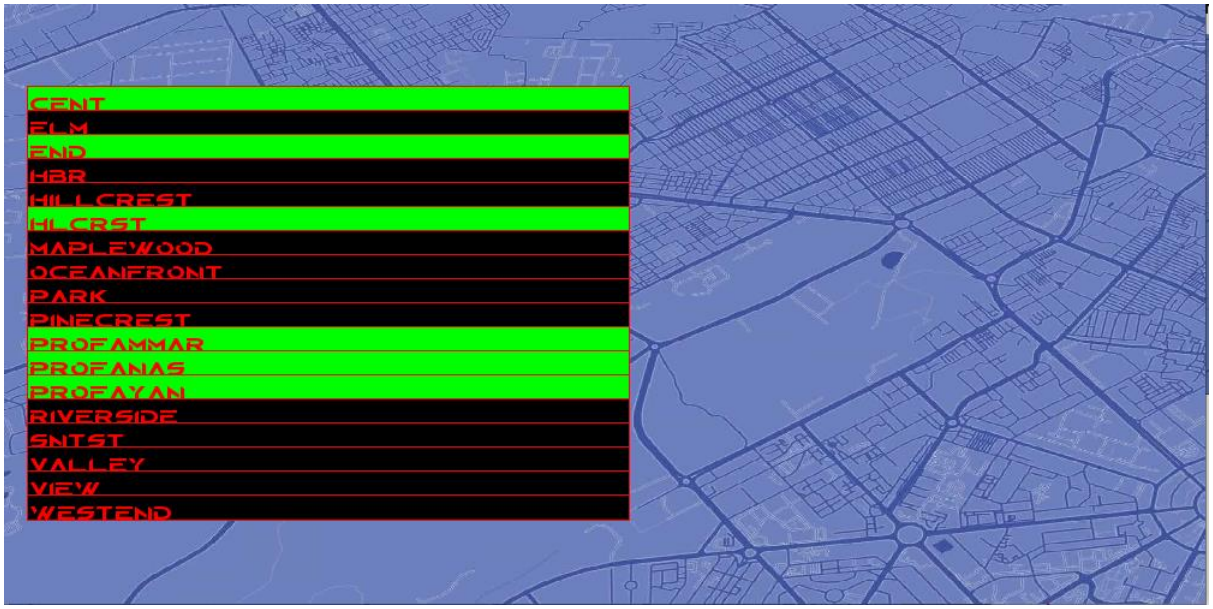


Adding a station:

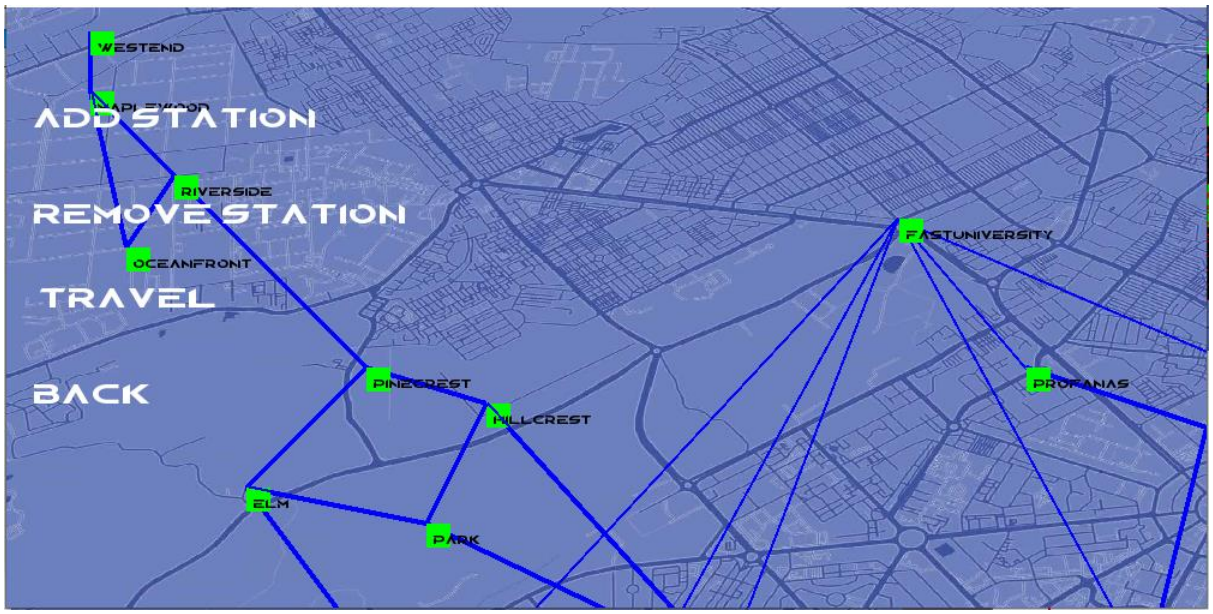


Choosing the location to add station:

Setting up links and paths with other stations:

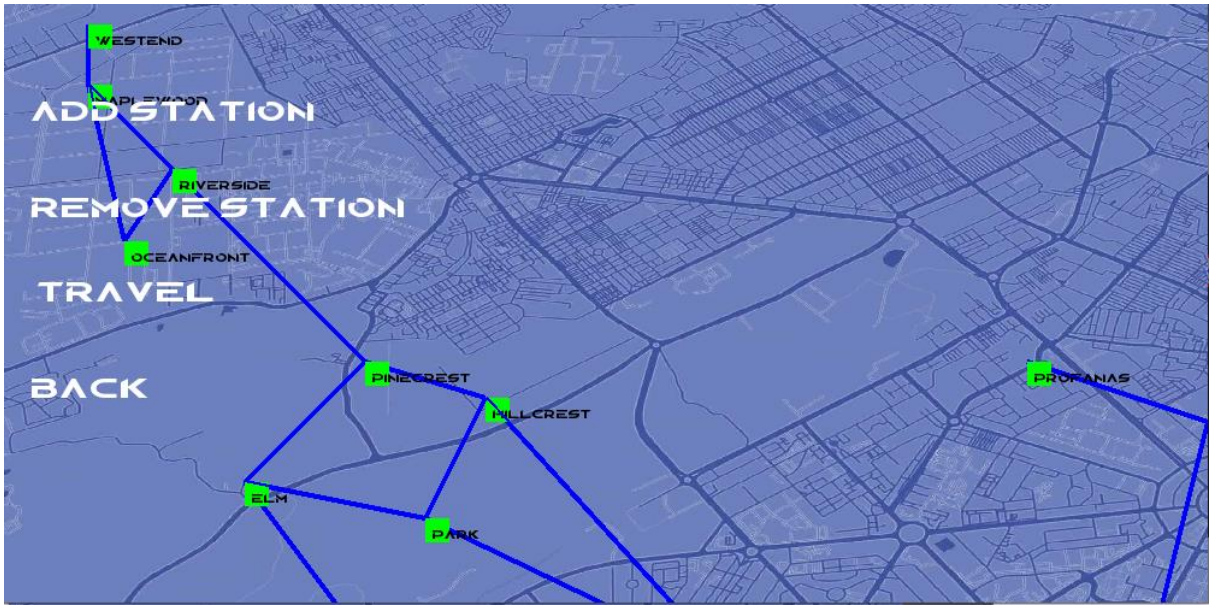


Showing the Result:



Now removing a station:

Map after the removal of vertex:

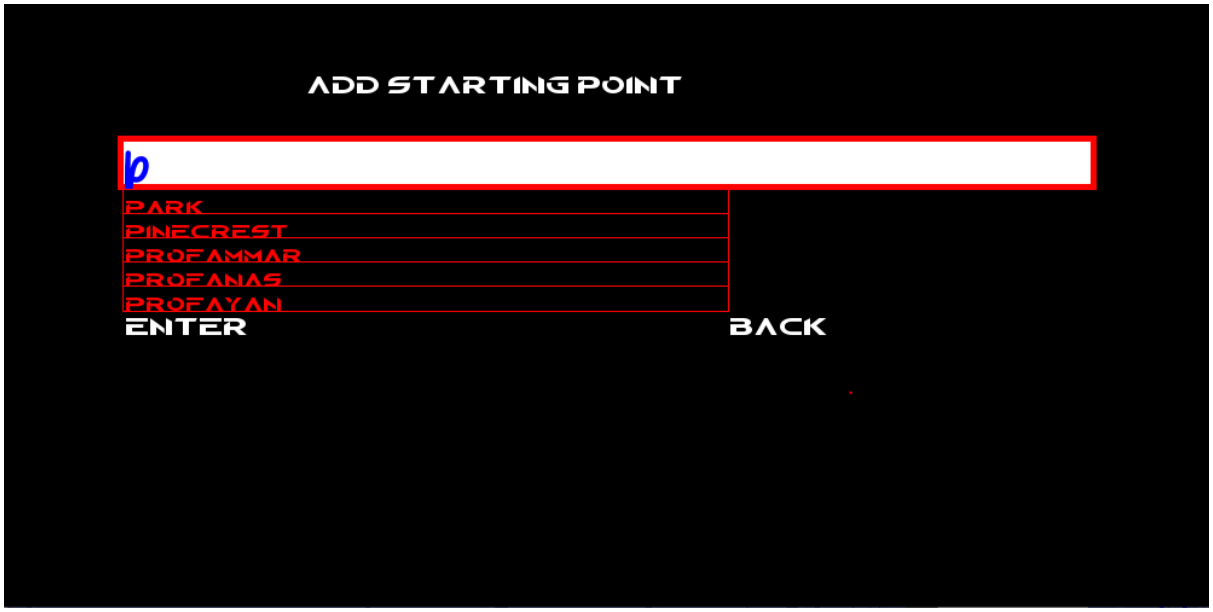


Second Menu, inside Travel:

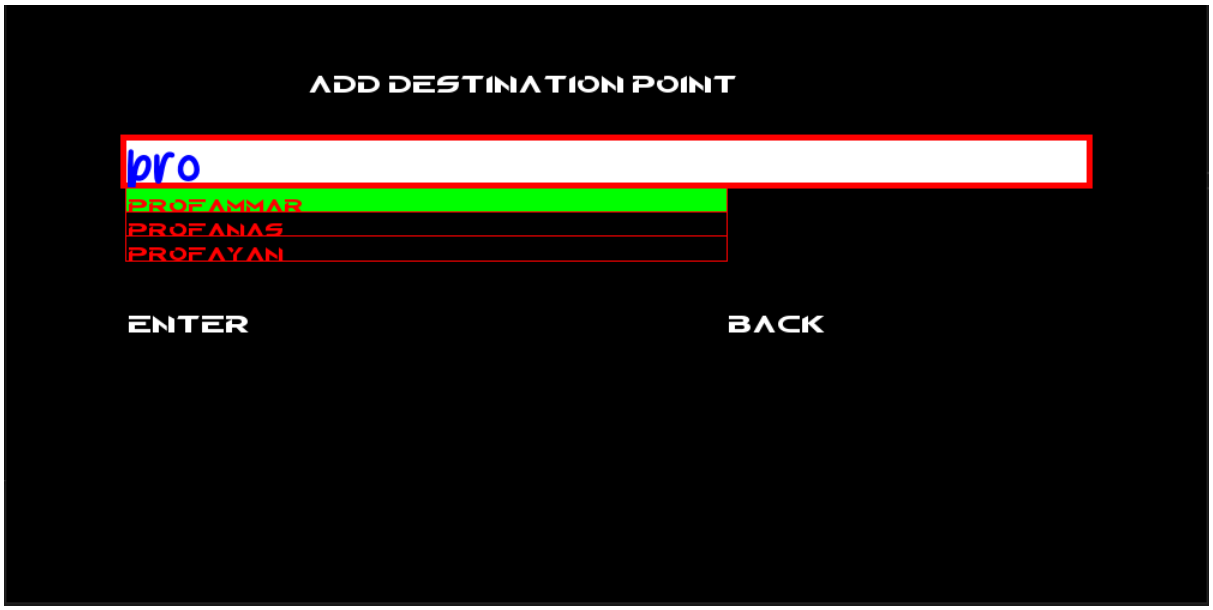
The description for these options is as follows:

- **Add Stop:** Allows you to enter a certain start location and end location, and calculates your fare price and total distance that you will be travelling by using Astar Search algorithm.
- **Remove Stop:** Allows you to remove a specific journey that you have travelled in your history.
- **Show Journey:** Shows your history and the paths that you have travelled along with the minimum distance and fare price.

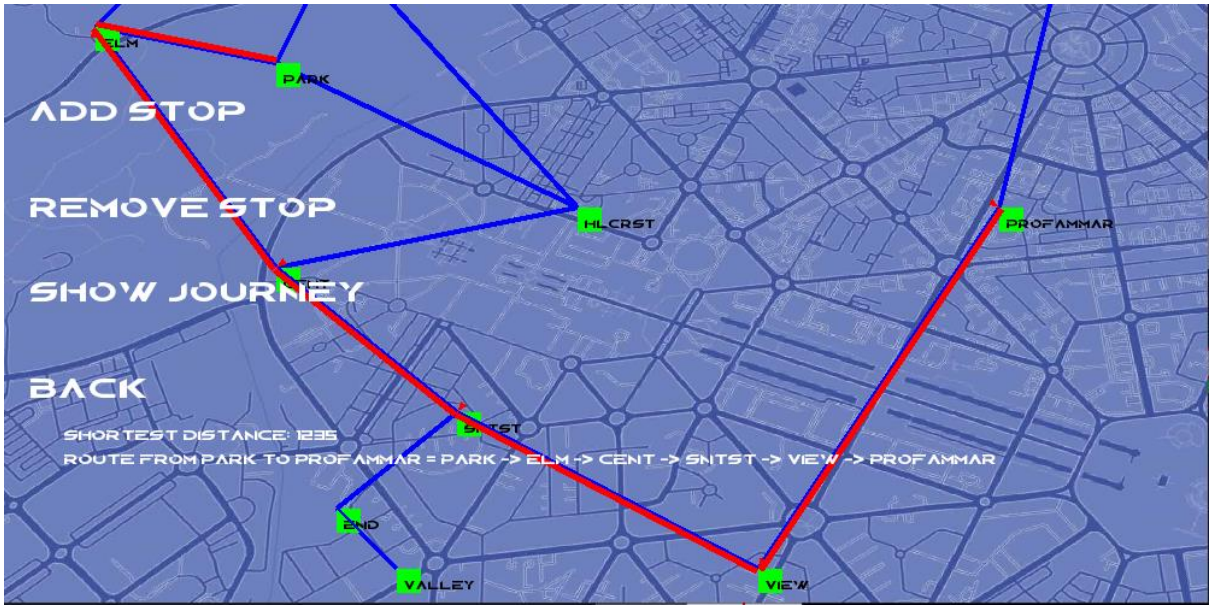
Adding starting point, to travel:



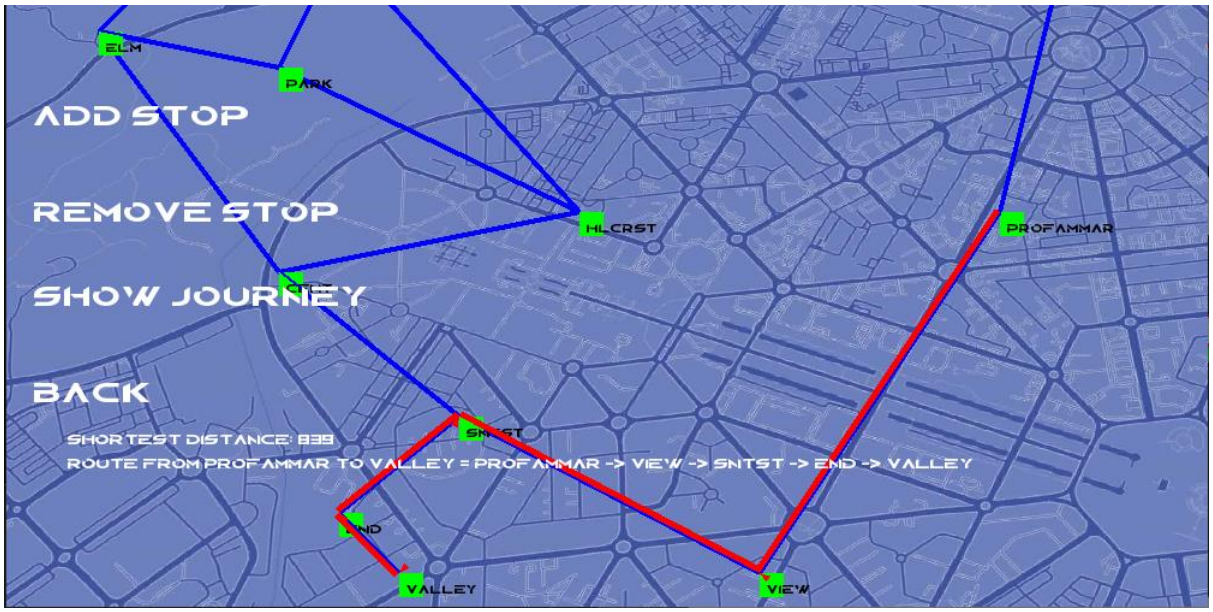
Adding destination point, to travel:



Shortest path from start to destination shown:

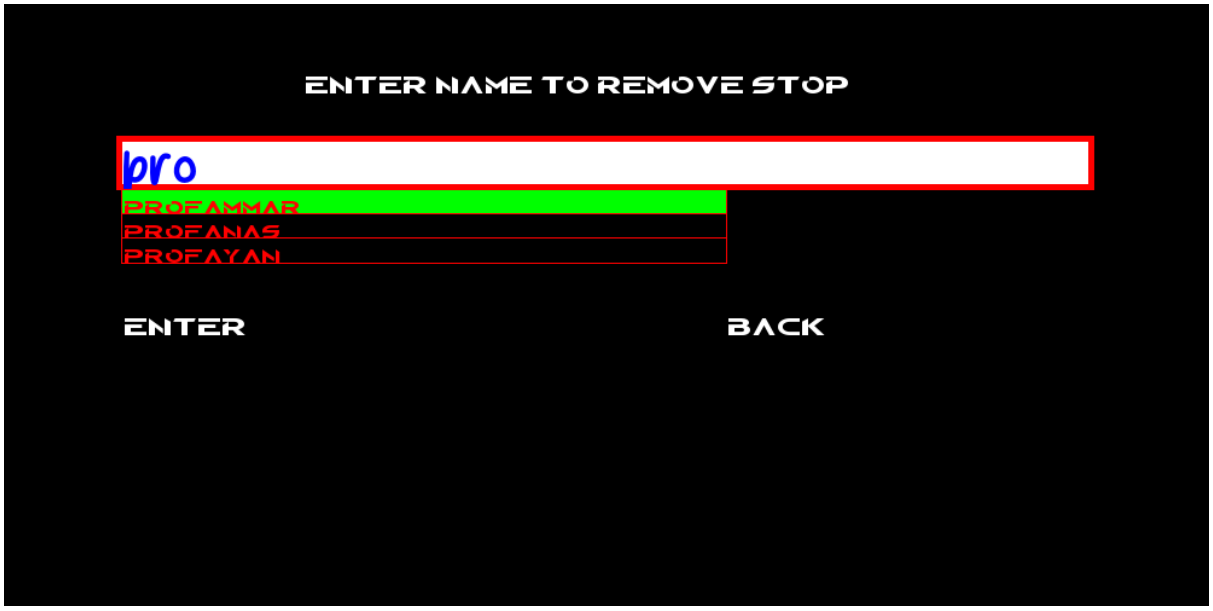


Adding a Stop again, to calculate distance and path from previous destination to current destination:



Showing the history:

Removing stop:



Shows the previous path visited: